

Music Playlist Manager

Software Architecture Design

GRUPPO 10

Alfonso Villani
Giovanni Lamberti
Luca Lanzetta
Nicola Liguori

User Stories

A partire dalle User Epics abbiamo:

- Definito le User Stories adottando il template:
 - **WHO** are we building it for?
 - **WHAT** is the intention?
 - **WHY** are we building it?
- Assegnato le **priorità** relative a ciascuna Story
- Determinato i **criteri di accettazione** per ogni storia
- Stimato l'effort assegnando gli **Story Points** ad ogni Story tramite sessioni di Planning Poker in team

○ US1 (Aggiunta Traccia)

+ Aggiungi

🕒 Date

☑ Checklist

👤 Membri

📎 Allegato

Etichette

Alta

+

Story points

5



Descrizione

Modifica

Come utente, **voglio** poter aggiungere una nuova canzone alla libreria inserendo *Titolo, Autore, Durata, Genere* e *Anno di pubblicazione*, **per** poterla ascoltare in futuro.

Acceptance Criteria:

- Il sistema mostra un form con campi per *Titolo, Autore, Durata, Genere* e *Anno*.
- *Titolo* e *Autore* sono campi obbligatori; il salvataggio fallisce se sono vuoti.
- Una volta salvata, la traccia compare immediatamente nella vista generale della libreria.



Activity Timer



▶ Start timer

+ Add time

🕒 0m

📅 Estimate: 0m

Product Backlog

ALTA

MEDIA

BASSA

US1 (Aggiunta Traccia)

≡ P 5

US2 (Modifica Traccia)

≡ P 2

US11 (Undo)

≡ P 5

US15 (Riordino Playlist Live)

≡ P 8

US3 (Elimina Traccia)

≡ P 3

US4 (Visualizzazione Tracce)

≡ P 3

US12 (Statistiche Home Page)

≡ P 3

US5 (Creazione Playlist)

≡ P 3

US6 (Aggiunta a Playlist)

≡ P 5

US13 (Tag Visivi)

≡ P 3

US7 (Rimozione da Playlist)

≡ P 5

US8 (Controlli Player)

≡ P 8

US14 (Auto Playlist)

≡ P 8

US9 (Aggiornamento UI)

≡ P 5

US10 (Modalità di Ascolto)

≡ P 5

71

Story Points
Totali

Sprint 1 Backlog

24 Story Points
Stimati

Privilegiare il rilascio di una base funzionante che integri le funzionalità principali (**Libreria**, **Playlist**, **Controlli**) rispetto alla completezza di una singola funzionalità

Alta Sprint 1
US1 (Aggiunta Traccia)
☰ ☑ 0/6 P 5

Alta Sprint 1
US5 (Creazione Playlist)
☰ ☑ 0/6 P 3

Alta Sprint 1
US8 (Controlli Player)
☰ ☑ 0/6 P 8

Alta Sprint 1
US4 (Visualizzazione Tracce)
☰ ☑ 0/6 P 3

Alta Sprint 1
US6 (Aggiunta a Playlist)
☰ ☑ 0/6 P 5

Definition of Done (DoD)

Una *user story* è **DONE** quando sono soddisfatti tutti gli elementi della checklist:

- Tutte le **task** della user story sono state completate
- L'implementazione della user story soddisfa **tutti** gli acceptance criteria
- **Unit Tests** scritti, eseguiti e superati
- L'implementazione della user story è stata **revisionata** da almeno un altro membro del team
- Il codice è stato opportunamente **commentato**
- Il codice è completo ed è stato committato su **GitHub**

Tools & Environment



Trello - Scrum Board
([Link](#))



Git Repository
([Link](#))



Documento Tecnico di Progetto
([Link](#))

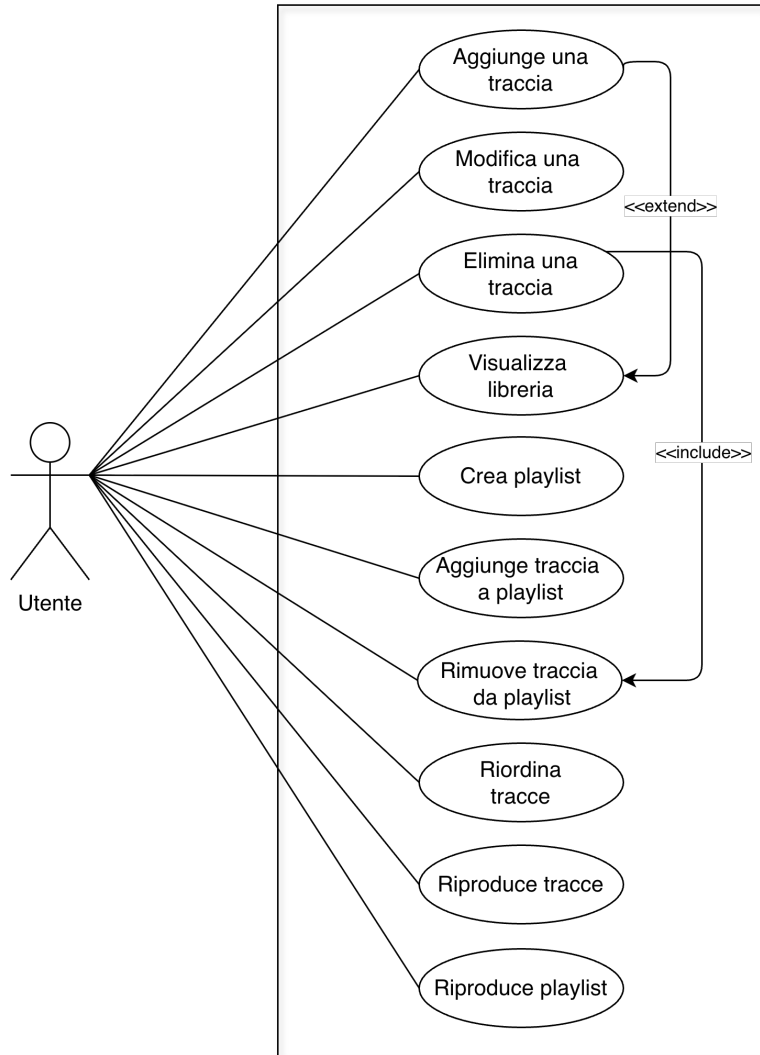


Java 17 con Maven
+ JavaFX

CODING CONVENTIONS

- Notazione **CamelCase** (Lower per variabili e funzioni, Upper per classi e interfacce)
- Codebase in inglese
- **Command-Query Separation**
- Commenti introduttivi per le classi

Software Estimation



$$UCW = 1 \times 3 = 3$$

$$UUCW = 10 \times 5 = 50$$

$$TCF = 0,745$$

$$UCF = 0,815$$

$$\Rightarrow \text{UCP} = (UUCW + UAW) \times TCF \times ECF = 32,18$$

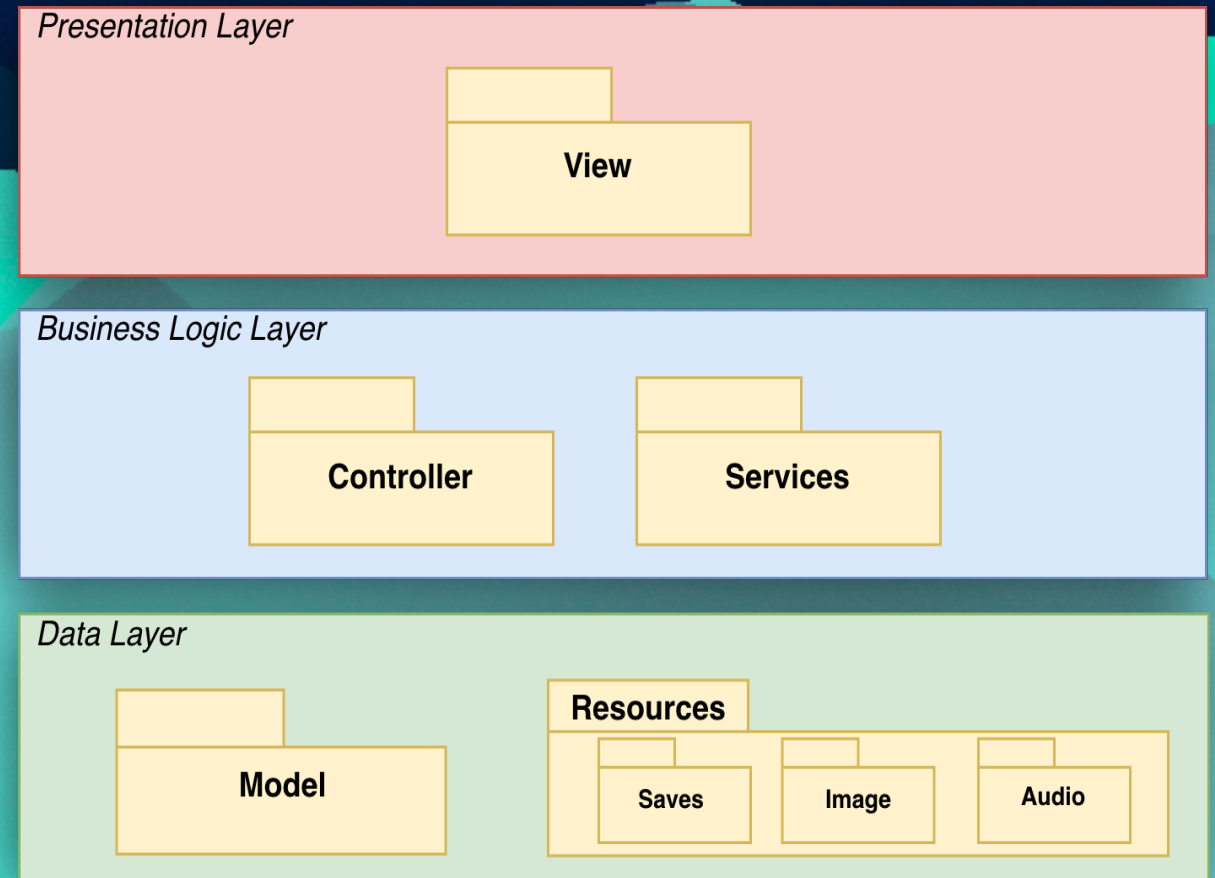
$$PROD = 10 \text{ ore/persona per UCP}$$

$$\Rightarrow \text{Effort} = PROD \times UCP \cong 322 \text{ ore/persona}$$

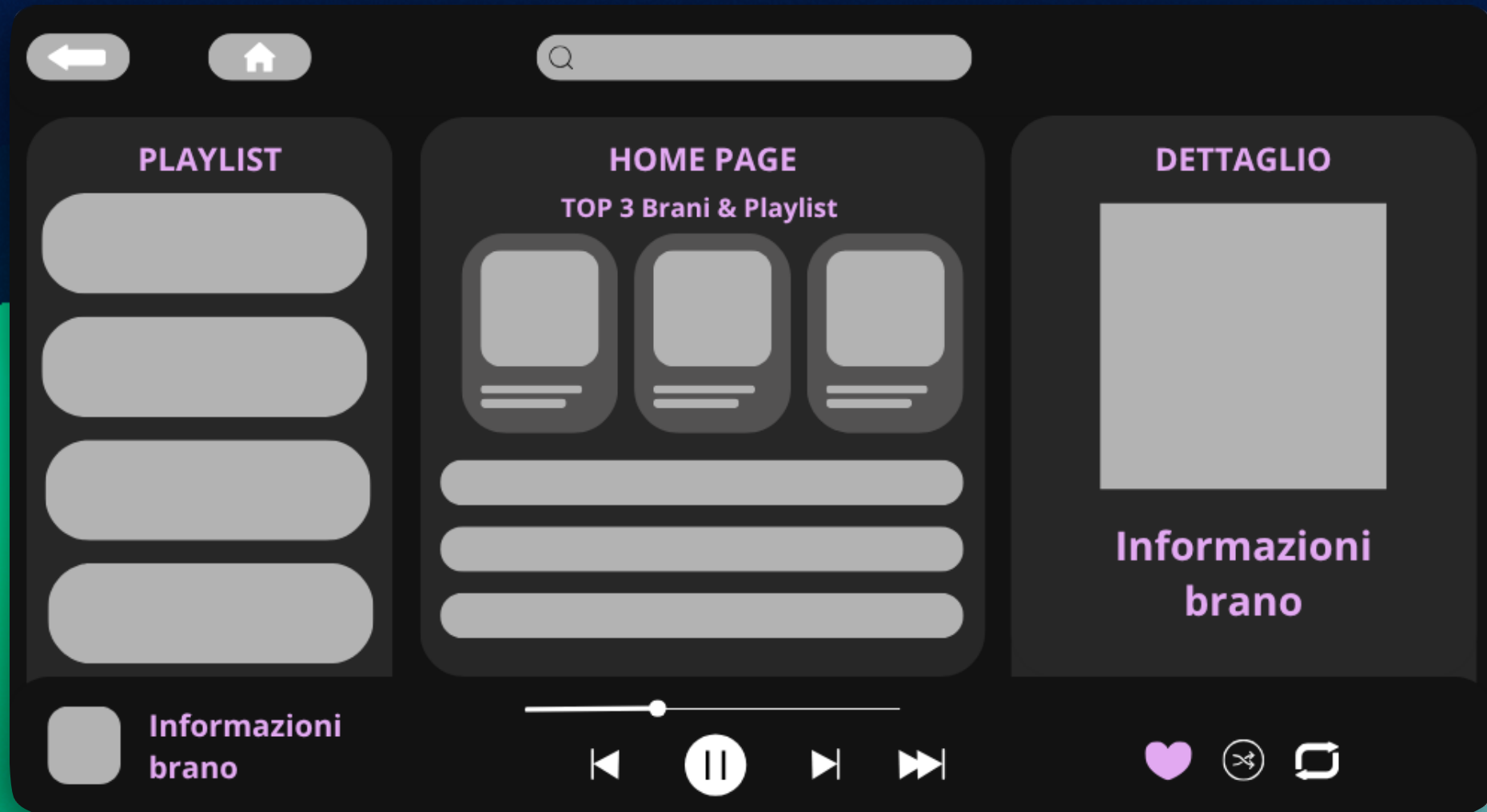
Scelta Architeturale: Pattern MVC

Perché abbiamo scelto il pattern **MVC**?

- **Separazione delle responsabilità** (dati, logica e UI) → alta coesione, basso accoppiamento
- **Compatibilità** → il framework di JavaFX supporta il pattern MVC nativamente
- **Testabilità** → la logica di business è testabile con JUnit indipendentemente dalla UI



View Mockup



Grazie per l'attenzione

